

Einfluss der Entwicklerkompetenz
auf die Effektivität von AI Coding
Agents:
Eine empirische Untersuchung von
Vibe-Coding in der
Unternehmenssoftwareentwicklung

Cihad Gök

Dezember 2025

Inhaltsverzeichnis

1	Einleitung	5
1.1	Motivation	5
1.2	Problemstellung	5
1.3	Zielsetzung der Arbeit	5
1.4	Aufbau der Arbeit	5
2	Theoretischer Hintergrund und Stand der Forschung	6
2.1	AI Coding Agents und das Paradigma des Vibe-Codings	6
2.1.1	Technologische Grundlagen: Transformer und In-Context Learning	6
2.1.2	Definition: Vibe-Coding als System-1-Aktivität	7
2.1.3	Theoretische Einordnung in die HCI-Forschung	7
2.2	Kompetenzmodelle: Vom Dreyfus-Modell zu kognitiven Architekturen	8
2.2.1	Das Dreyfus-Modell der Expertise	8
2.2.2	Kritik und Erweiterung: ACT-R und Adaptive Expertise	8
2.2.3	Situated Learning und Cognitive Apprenticeship	9
2.3	Kognitionspsychologische Aspekte der Human-AI Interaction	9
2.3.1	Automation Bias und Vertrauen	9
2.3.2	Cognitive Load Theory (CLT)	9
2.3.3	Mixed-Initiative Interaction	10
2.4	Empirische Studien zu KI-Assistenz	10
2.5	Softwarequalität und Produktivität	10
2.5.1	Softwarequalität nach ISO/IEC 25010	10
2.5.2	Produktivität: Das SPACE-Framework	11
2.6	Zusammenfassung und Forschungslücke	11
3	Forschungsdesign und Hypothesenentwicklung	12
3.1	Methodischer Ansatz: Between-Subjects Design	12
3.2	Theoretische Herleitung der Hypothesen	12
3.2.1	H1: Der „Sweet Spot“ der KI-Unterstützung (Mid-Code)	12
3.2.2	H2: Das Validierungs-Paradoxon (Low-Code)	13
3.2.3	H3: Skeptizismus und Qualitätsfokus (High-Code)	13
3.3	Operationalisierung der Kompetenzstufen	13
3.4	Kontrolle von Störvariablen (Confounder)	13
3.5	Die Testumgebung als unabhängige Variable: Ecological Validity	14
4	Durchführung und Methodik der Datenerhebung	15
4.1	Ablauf der Untersuchung	15

4.2	Operationalisierung der Metriken	16
4.2.1	Funktionale Korrektheit (Effectiveness)	16
4.2.2	Effizienz (Efficiency)	16
4.2.3	Codequalität (Quality)	16
4.2.4	Interaktionsqualität (Interaction Quality)	16
4.3	Güte der Untersuchung (Threats to Validity)	17
4.3.1	Konstruktvalidität (Construct Validity)	17
4.3.2	Interne Validität (Internal Validity)	17
4.3.3	Externe Validität (External Validity)	17
4.3.4	Reliabilität (Reliability)	18
4.4	Datenanalyse-Strategie	18
5	Technische Realisierung der Testumgebung	19
5.1	Anforderungen an die Sandbox	19
5.2	Architektur und Tech-Stack: Kognitive Dimensionen	19
5.2.1	Backend: Node.js & Express (Layered Architecture)	19
5.2.2	Frontend: React & Vite	20
5.3	Technische Funktionsweise von Copilot in der Sandbox	20
5.3.1	Prompt Construction und Context Retrieval	20
5.3.2	Token Prediction und Ranking	21
5.4	Design der Aufgaben (Workload Grading)	21
5.5	Automatisierung der Evaluation	21
5.5.1	Das Analyse-Skript (<code>run.js</code>)	21
5.5.2	Grenzen der LLM-gestützten Code-Analyse	22
5.6	Relevanz für den Unternehmenskontext (BuyIn)	22
5.7	Containerisierung und Reproduzierbarkeit	22
6	Ergebnisse	23
6.1	Datengrundlage und Toolkontext	23
6.1.1	Studiensetting und Rahmenbedingungen	23
6.1.2	Datenquellen und Artefakte	23
6.1.3	Dateninventar und Vollständigkeit	24
6.1.4	Zuordnung und Zusammenführung der Daten	24
6.1.5	Reproduzierbarkeit und Auswertungswerkzeuge	24
6.1.6	Abgrenzung zu Kapitel 7	25
6.2	Task-Pipeline-Analyse	25
6.3	Detaillierte Task-Analyse	25
6.3.1	Task 1: Persistente Datenspeicherung	25
6.3.2	Task 2: Benutzerauthentifizierung	25
6.3.3	Task 3: Kategorien und Farbcodierung	25
6.3.4	Task 4: Dateianhänge	26
6.3.5	Task 5: Kalenderansicht	26
6.4	Code-Qualität und Metriken	26
6.5	Chat-Interaktionsmuster	26

6.6	Triangulation der Datenquellen	26
6.7	Zusammenfassung der Ergebnisse	26

1 Einleitung

1.1 Motivation

Die Softwareentwicklung befindet sich in einem fundamentalen Wandel. Durch den Einsatz von *Large Language Models* (LLMs) und darauf basierenden *AI Coding Agents* verändert sich die Art und Weise, wie Software konzipiert, implementiert und gewartet wird.

1.2 Problemstellung

Unternehmen stehen vor der Herausforderung, diese neuen Werkzeuge produktiv und sicher in ihre bestehenden Entwicklungsprozesse zu integrieren. Dabei ist unklar, wie sich der Einsatz von AI Coding Agents auf die Codequalität, die Entwicklerproduktivität und die Sicherheit auswirkt.

1.3 Zielsetzung der Arbeit

Ziel dieser Arbeit ist es, verschiedene AI Coding Agents zu bewerten und eine Integrationsstrategie für das Unternehmen BuyIn zu entwickeln. Dabei wird das Konzept des *Vibe-Coding* als neuer Entwicklungsstil untersucht.

1.4 Aufbau der Arbeit

Die Arbeit gliedert sich in folgende Kapitel:

- Kapitel 2: Hintergrund und Stand der Forschung
- Kapitel 3: Forschungsdesign
- Kapitel 4: Methodik
- Kapitel 5: Implementierung der Sandbox
- ...

2 Theoretischer Hintergrund und Stand der Forschung

Dieses Kapitel legt das theoretische Fundament für die Untersuchung der Interaktion zwischen menschlicher Kompetenz und KI-gestützter Code-Generierung. Es definiert zunächst das Konzept der *AI Coding Agents* und grenzt den neuartigen Arbeitsmodus des *Vibe-Codings* von klassischer Softwareentwicklung ab (Abschnitt 2.1). Darauf aufbauend wird ein Kompetenzmodell eingeführt, das auf dem Dreyfus-Modell der Expertise basiert, jedoch kritisch um moderne kognitive Architekturen wie ACT-R erweitert wird (Abschnitt 2.2). Anschließend werden kognitionspsychologische Aspekte der Mensch-Maschine-Interaktion – insbesondere *Dual-Process Theory*, *Cognitive Load* und *Automation Bias* – beleuchtet, die erklären, warum unterschiedliche Kompetenzstufen unterschiedlich stark von KI profitieren (Abschnitt 2.3). Abschließend werden Qualitäts- und Produktivitätsmodelle (ISO 25010, SPACE) vorgestellt, die als Messinstrumente dienen (Abschnitt 2.5).

2.1 AI Coding Agents und das Paradigma des Vibe-Codings

Die Integration von *Large Language Models (LLMs)* in die Softwareentwicklung markiert einen Paradigmenwechsel. Während klassische IDE-Features wie IntelliSense deterministisch und syntaktisch operieren, arbeiten AI Coding Agents probabilistisch und semantisch.

2.1.1 Technologische Grundlagen: Transformer und In-Context Learning

Moderne AI Coding Agents wie **GitHub Copilot** basieren auf der Transformer-Architektur (Vaswani u. a. 2017), die es ermöglicht, Abhängigkeiten in Sequenzen über lange Distanzen zu modellieren. Technisch relevant für die Interaktion ist das *Fill-In-The-Middle (FIM)* Paradigma (Bavarian u. a. 2022). Das Modell sagt nicht nur das nächste Token voraus, sondern berücksichtigt den Kontext *vor* und *nach* der Cursor-Position.

Zusätzlich nutzen diese Systeme *Retrieval-Augmented Generation (RAG)*-ähnliche Mechanismen. Der Agent scannt den lokalen Kontext (geöffnete Tabs, importierte Module) und injiziert relevante Code-Snippets in den Prompt, ohne dass der Nutzer dies explizit steuert. Dies reduziert die Notwendigkeit für manuelles Prompt Engineering,

erhöht aber die Abhängigkeit von einer sauberen Projektstruktur, da der Agent nur so gut ist wie der Kontext, den er „sieht“ (Witte u. a. 2023).

2.1.2 Definition: Vibe-Coding als System-1-Aktivität

Der Begriff *Vibe-Coding* beschreibt einen emergenten Entwicklungsstil, bei dem der Fokus von der imperativen Syntax-Erstellung zur deklarativen Intentions-Beschreibung verschiebt. Ich definiere Vibe-Coding für diese Arbeit als:

„Einen iterativen Prozess der Softwareerstellung, bei dem der Mensch primär als Architekt und Reviewer agiert, während die KI die Implementierungsdetails auf Basis von natürlichsprachlichen oder kontextuellen ‚Vibes‘ (Intentionen) generiert.“

Dieser Modus lässt sich durch die *Dual-Process Theory* von Kahneman (Kahneman 2011) erklären:

- **System 1 (Schnelles Denken):** Vibe-Coding appelliert an die Intuition. Der Entwickler tippt einen Funktionsnamen, der Agent schlägt den Body vor. Die Akzeptanz erfolgt oft heuristisch („Sieht gut aus“).
- **System 2 (Langsames Denken):** Die Validierung des Codes erfordert analytische Anstrengung. Hier liegt die Gefahr: Wenn der Agent (System 1) zu gut wirkt, wird System 2 nicht aktiviert, was zu Fehlern führt.

Dieser Shift verändert die Anforderungen fundamental:

- **Shift from Writing to Reading:** Der Anteil des Code-Lesens und -Verstehens steigt massiv an.
- **Shift from Syntax to Semantics:** Entwickler:innen müssen weniger wissen, *wie* eine Schleife syntaktisch korrekt ist, aber präziser verstehen, *was* sie logisch bewirken soll.

2.1.3 Theoretische Einordnung in die HCI-Forschung

In der Human-Computer Interaction (HCI) lässt sich Vibe-Coding als eine Form der *End-User Development* (EUD) verstehen, bei der die Barriere zur Code-Erstellung gesenkt wird. Es weist Parallelen zum *Natural Language Programming* auf, unterscheidet sich jedoch durch den *Mixed-Initiative*-Charakter (Horvitz 1999): Nicht nur der Mensch gibt Befehle, sondern die KI schlägt proaktiv Lösungen vor (*Ghost Text*). Dies erfordert eine ständige Synchronisation der mentalen Modelle zwischen Mensch und Maschine.

2.2 Kompetenzmodelle: Vom Dreyfus-Modell zu kognitiven Architekturen

Um den Einfluss der menschlichen Kompetenz zu messen, ist eine fundierte Klassifizierung notwendig.

2.2.1 Das Dreyfus-Modell der Expertise

Das *Dreyfus-Modell der Expertise* (H. L. Dreyfus und S. E. Dreyfus 1986) beschreibt den Erwerb von Fähigkeiten in fünf Stufen, von der strikten Regelbefolgung bis zur intuitiven Meisterschaft. Für diese Arbeit werden vier Stufen adaptiert:

1. **Novice (No-Code):** Befolgt strikte Regeln, hat kein kontextuelles Verständnis. Verlässt sich vollständig auf die KI als „Black Box“.
2. **Advanced Beginner (Low-Code):** Kann einfache Aufgaben lösen, erkennt aber keine übergeordneten Muster. Kann KI-Vorschläge syntaktisch, aber oft nicht semantisch bewerten.
3. **Competent (Mid-Code):** Arbeitet zielorientiert und kann Probleme in Teilprobleme zerlegen. Nutzt KI als Werkzeug zur Beschleunigung, besitzt aber genug Modellverständnis zur Validierung.
4. **Proficient/Expert (High-Code):** Handelt intuitiv und erkennt Abweichungen sofort. Nutzt KI zur Automatisierung von Boilerplate, ist aber skeptisch gegenüber komplexer Logik.

2.2.2 Kritik und Erweiterung: ACT-R und Adaptive Expertise

Das Dreyfus-Modell wird oft als zu linear kritisiert. Es vernachlässigt die *adaptive Expertise* (Hatano & Inagaki), also die Fähigkeit, Wissen auf neue Domänen zu übertragen. Hier bietet die kognitive Architektur **ACT-R** (Adaptive Control of Thought-Rational) von Anderson (1996) eine tiefere Erklärung:

- **Deklaratives Wissen:** Faktenwissen („Was ist eine Schleife?“). Novizen müssen dieses Wissen mühsam abrufen.
- **Prozedurales Wissen:** Handlungswissen („Wie schreibe ich eine Schleife?“). Experten haben dies in „Produktionsregeln“ kompiliert, die schnell und mühelos ablaufen.

Relevanz für Copilot: Der Agent liefert *prozedurale Chunks* (fertigen Code). Für Novizen, die nur über deklaratives Wissen verfügen, wirkt dies wie eine „Prothese“, die fehlende Produktionsregeln ersetzt. Für Experten ist es eine Beschleunigung bereits vorhandener Regeln.

2.2.3 Situated Learning und Cognitive Apprenticeship

Ergänzend bietet der Ansatz des *Situated Learning* (Lave und Wenger 1991) und *Cognitive Apprenticeship* (Collins u. a. 1989) eine Perspektive auf das Lernen mit KI.

- **Scaffolding:** Der Agent dient als „Gerüst“ (Scaffold), das es Lernenden ermöglicht, Aufgaben zu lösen, die knapp über ihrem aktuellen Kompetenzniveau liegen (Zone der nächsten Entwicklung).
- **Legitimate Peripheral Participation:** Novizen können durch KI-Unterstützung früher an produktiven Prozessen teilnehmen, auch wenn sie die volle Komplexität noch nicht durchdringen.

2.3 Kognitionspsychologische Aspekte der Human-AI Interaction

Der Erfolg von AI Coding Agents hängt maßgeblich von der menschlichen Komponente ab. Theorien der Mensch-Maschine-Interaktion erklären, warum Nutzende unterschiedlich auf KI-Vorschläge reagieren.

2.3.1 Automation Bias und Vertrauen

Ein zentrales Phänomen ist der *Automation Bias* (Parasuraman und Manzey 2010): Die Tendenz, automatisierten Systemen mehr zu vertrauen als der eigenen Urteilkraft. Dies manifestiert sich in zwei Fehlerarten:

- **Errors of Commission:** Der Nutzer akzeptiert einen falschen Vorschlag der KI (häufig bei Low-Code).
- **Errors of Omission:** Der Nutzer übersieht einen Fehler, weil das System keine Warnung ausgibt.

Das Vertrauen (*Trust*) muss kalibriert sein (Lee und See 2004). *Overtrust* führt zu unkritischer Übernahme, *Distrust* (oft bei Experten) zu ineffizienter manueller Überprüfung.

2.3.2 Cognitive Load Theory (CLT)

Die *Cognitive Load Theory* (Sweller 1988) unterscheidet drei Arten der kognitiven Belastung:

- **Intrinsic Load:** Die inhärente Komplexität der Aufgabe (z.B. Rekursion verstehen).
- **Extraneous Load:** Belastung durch die Darstellung oder das Werkzeug (z.B. komplexe IDE-Menüs).

- **Germane Load:** Die kognitive Kapazität, die für das Lernen und Schema-Bildung genutzt wird.

AI Coding Agents haben das Potenzial, den *Extraneous Load* (Syntax-Details) massiv zu senken. Dies setzt jedoch *Germane Load* frei, die für die Validierung genutzt werden muss. Wenn Novizen keine mentalen Modelle (Schemata) haben, können sie diese freigewordene Kapazität nicht nutzen, und der *Intrinsic Load* der Validierung überfordert sie. **Kritik:** Die CLT ist schwer in Echtzeit zu messen. Dennoch bietet sie ein robustes Erklärungsmodell für die Überforderung von Novizen trotz KI-Hilfe.

2.3.3 Mixed-Initiative Interaction

Die Interaktion mit Copilot ist ein Beispiel für *Mixed-Initiative Interaction* (Horvitz 1999). Beide Parteien (Mensch und Agent) können die Initiative ergreifen. Der Agent schlägt proaktiv Code vor (Ghost Text), der Mensch kann diesen annehmen, ablehnen oder durch Weitertippen modifizieren. Die Effizienz dieser Kollaboration hängt davon ab, wie gut der Mensch die Intention des Agenten vorhersagen und steuern kann (*Theory of Mind* für die KI).

2.4 Empirische Studien zu KI-Assistenz

Aktuelle Studien bestätigen die theoretischen Annahmen teilweise, zeigen aber auch neue Phänomene:

- **Grounded Copilot:** Barke u. a. (2023) identifizierten zwei Modi der Interaktion: *Acceleration* (schnelleres Tippen bekannter Logik) und *Exploration* (Nutzen der KI, um neue APIs zu entdecken).
- **Reading Between the Lines:** Mozannar u. a. (2022) zeigten, dass das Modifizieren von KI-Code oft kognitiv teurer ist als das Selbstschreiben, wenn der Vorschlag subtile Fehler enthält.

2.5 Softwarequalität und Produktivität

Die Bewertung der Arbeit mit AI Coding Agents erfordert klare Kriterien für Qualität und Produktivität.

2.5.1 Softwarequalität nach ISO/IEC 25010

Der Standard ISO/IEC 25010 definiert Softwarequalität über acht Hauptmerkmale (*ISO/IEC 25010:2011 Systems and Software Engineering—System and Software Quality Models* 2011). Für die Bewertung von KI-generiertem Code sind insbesondere relevant:

- **Funktionale Eignung:** Erfüllt der generierte Code die fachlichen Anforderungen korrekt?

- **Wartbarkeit:** Ist der Code modular, verständlich und konsistent strukturiert? Studien zeigen, dass KI-Tools dazu verleiten können, Code zu duplizieren statt zu abstrahieren, was die Wartbarkeit langfristig gefährdet (Nadi u. a. 2022).
- **Sicherheit:** Enthält der Code Schwachstellen? Da LLMs auf öffentlichen Daten trainiert sind, können sie unsichere Muster reproduzieren (Pearce u. a. 2022).

2.5.2 Produktivität: Das SPACE-Framework

Produktivität in der Softwareentwicklung ist mehrdimensional. Das SPACE-Framework (Forsgren u. a. 2021) unterscheidet fünf Dimensionen, die auch für den Einsatz von Copilot zentral sind:

- **S (Satisfaction):** Steigert das Tool die Zufriedenheit und reduziert es Frustration?
- **P (Performance):** Führt der Einsatz zu qualitativ besseren Ergebnissen?
- **A (Activity):** Wie viele Aufgaben können in einer bestimmten Zeit abgeschlossen werden?
- **C (Communication):** Wie verändert sich die Zusammenarbeit (hier: Mensch-Maschine-Interaktion)?
- **E (Efficiency & Flow):** Wird der Arbeitsfluss durch unterbrechungsfreie Vorschläge gefördert?

2.6 Zusammenfassung und Forschungslücke

Bisherige Studien konzentrierten sich oft auf die technische Leistungsfähigkeit von Modellen oder allgemeine Produktivitätsgewinne (Peng u. a. 2023). Es fehlt jedoch an einer differenzierten Betrachtung, wie die *individuelle Kompetenz* (operationalisiert durch das Dreyfus-Modell und ACT-R) die Interaktion mit dem Agenten moderiert. Insbesondere die Frage, ob KI als „Equalizer“ wirkt (Novizen auf Experten-Niveau hebt) oder die Kluft vergrößert („Rich get richer“), ist ungeklärt. Diese Arbeit adressiert diese Lücke durch ein experimentelles Design, das kognitive Theorien (Dual-Process, CLT) mit empirischen Software-Engineering-Metriken verbindet.

3 Forschungsdesign und Hypothesenentwicklung

Dieses Kapitel leitet das empirische Design der Studie aus den in Kapitel 2 vorgestellten Theorien ab. Ziel ist es, den Einfluss der unabhängigen Variable *Kompetenz* auf die abhängigen Variablen *Qualität*, *Effizienz* und *Interaktion* in einer kontrollierten Brownfield-Umgebung zu isolieren.

3.1 Methodischer Ansatz: Between-Subjects Design

Die Untersuchung folgt einem *Between-Subjects Design*, bei dem Teilnehmende basierend auf ihrer Vorkenntnis in disjunkte Gruppen eingeteilt werden. **Begründung nach Wohlin u. a. (2012):**

- **Vermeidung von Lerneffekten (Carry-Over Effects):** Ein Within-Subjects Design (jede Person löst Aufgaben einmal mit, einmal ohne KI) wäre hier problematisch, da das Lösen einer Aufgabe Wissen für den zweiten Durchgang generiert.
- **Isolierung der Kompetenzvariable:** Das Ziel ist der Vergleich der Interaktionsmuster zwischen verschiedenen Kompetenzniveaus („Mensch A + Maschine“ vs. „Mensch B + Maschine“), nicht der Vergleich des Tools an sich.

3.2 Theoretische Herleitung der Hypothesen

Die Hypothesen werden direkt aus dem Dreyfus-Modell, der Dual-Process Theory und der Cognitive Load Theory (siehe Abschnitt 2.2 und 2.3) abgeleitet.

3.2.1 H1: Der „Sweet Spot“ der KI-Unterstützung (Mid-Code)

Theoretische Begründung: Nach der Cognitive Load Theory profitieren Lernende am meisten, wenn *Extraneous Load* (Syntax) reduziert wird, solange genug Schema-Wissen vorhanden ist, um den Output zu validieren. Mid-Coder befinden sich in einer Phase, in der sie Konzepte verstehen, aber die syntaktische Umsetzung noch kognitive Ressourcen bindet. Copilot übernimmt diese Last, wodurch *Germane Load* für architektonische Entscheidungen frei wird. **Alternative Hypothese:** Die „Rising Tide“-Hypothese würde besagen, dass alle Gruppen gleichermaßen profitieren. Dies widerspricht jedoch der Theorie der *Zone der nächsten Entwicklung*, da Novizen ohne Scaffolding überfordert sind.

Hypothese 1. *Teilnehmende der Gruppe **Mid-Code** erzielen den höchsten relativen Produktivitätsgewinn und die ausgewogenste Balance aus Geschwindigkeit und Qualität. Sie besitzen genug Expertise, um Automation Bias zu vermeiden, profitieren aber signifikant von der syntaktischen Entlastung.*

3.2.2 H2: Das Validierungs-Paradoxon (Low-Code)

Theoretische Begründung: Novizen und Advanced Beginners (Dreyfus Stufe 1–2) fehlt das mentale Modell, um subtile Fehler zu erkennen. Sie operieren primär im *System 1* (intuitiv) und akzeptieren plausibel klingende Vorschläge unkritisch (*Errors of Commission*). Da ihnen die Schemata fehlen, um den Code zu „parsen“, ist der *Intrinsic Load* der Validierung zu hoch. **Empirische Stützung:** Studien von Vaithilingam u. a. (2022) zeigen, dass Nutzer oft Code akzeptieren, den sie nicht verstehen, was zu schwer findbaren Bugs führt.

Hypothese 2. *Teilnehmende der Gruppen **No-Code** und **Low-Code** erreichen zwar funktionale Lösungen („es läuft“), weisen aber signifikant mehr Sicherheitslücken und Wartbarkeitsprobleme auf als höhere Gruppen, da sie Code unkritisch übernehmen (Overtrust).*

3.2.3 H3: Skeptizismus und Qualitätsfokus (High-Code)

Theoretische Begründung: Experten (Dreyfus Stufe 4–5) haben starke mentale Modelle und vertrauen der Automation weniger. Sie aktivieren *System 2* (analytisch) häufiger, um Vorschläge zu prüfen. Dies führt zu einem *Distrust*-Verhalten, bei dem Zeiterparnisse durch manuelle Reviews teilweise negiert werden. **Alternative Sichtweise:** Man könnte annehmen, Experten seien am schnellsten. Die Theorie der *Expertise Reversal Effect* (Kalyuga et al.) legt jedoch nahe, dass Hilfestellungen für Experten redundant und störend sein können.

Hypothese 3. ***High-Coder** nutzen den Agenten selektiver (weniger „Accept All“), was zu einer geringeren reinen Codier-Geschwindigkeit im Vergleich zu Mid-Codern führen kann, aber die höchste Codequalität (Wartbarkeit, Sicherheit) sicherstellt.*

3.3 Operationalisierung der Kompetenzstufen

Die Einteilung erfolgt nicht willkürlich, sondern basierend auf einem Scoring-Modell (0–13 Punkte), das die Dimensionen des Dreyfus-Modells abbildet.

3.4 Kontrolle von Störvariablen (Confounder)

Um die interne Validität zu sichern, wurden folgende Störvariablen kontrolliert:

- **IDE-Familiarity:** Alle Teilnehmenden nutzten VS Code. Unterschiede in der Vertrautheit wurden im Pre-Survey erfasst.

Gruppe	Dreyfus-Level	Score	Charakteristik
No-Code	Novice	0	Kein mentales Modell von Programmierung. Verlässt sich zu 100% auf KI als Black Box.
Low-Code	Advanced Beginner	1–5	Kennt Grundkonzepte, aber keine Architekturmuster. Hohes Risiko für <i>Spaghetti-Code</i> und <i>Automation Bias</i> .
Mid-Code	Competent	6–10	Versteht Zusammenhänge, plant Schritte. Kann KI-Fehler meist erkennen (<i>System 2</i> Aktivierung möglich).
High-Code	Proficient/Expert	>10	Intuitives Verständnis. Nutzt KI zur Beschleunigung von Boilerplate, nicht zur Lösungsfindung.

Tabelle 3.1: Mapping von Kompetenzgruppen auf das Dreyfus-Modell

- **Domain Knowledge:** Das Todo-App-Szenario ist generisch genug, um kein spezifisches Domänenwissen (wie z.B. in der Medizin) vorauszusetzen.
- **Englischkenntnisse:** Da Copilot primär auf englischem Code trainiert ist, wurden Grundkenntnisse vorausgesetzt.

3.5 Die Testumgebung als unabhängige Variable: Ecological Validity

Die Wahl einer **Brownfield-Umgebung** (bestehender Code) statt Greenfield (leeres Blatt) ist essenziell für die *ökologische Validität* der Studie. In der industriellen Praxis arbeiten Entwickler:innen zu ca. 90% in bestehenden Systemen (Brandtner u. a. 2024).

- **Kontext-Sensitivität (RAG-Test):** Der Agent muss den bestehenden Style (z.B. SCSS-Module, React Hooks) erkennen und adaptieren. Dies testet die Fähigkeit des Modells, Kontext zu nutzen, und die Fähigkeit des Nutzers, diesen Kontext bereitzustellen (z.B. durch Öffnen relevanter Dateien).
- **Kognitive Belastung:** Die Teilnehmenden müssen sich in der Struktur zurechtfinden, was für Low-Coder eine zusätzliche Hürde darstellt (*Intrinsic Load*). Greenfield-Aufgaben (z.B. LeetCode) würden diese Komplexität ausblenden und die Ergebnisse verzerren.

Dies stellt sicher, dass wir *reale* Arbeitsbedingungen simulieren und nicht nur die Fähigkeit testen, isolierte Algorithmen zu generieren.

4 Durchführung und Methodik der Datenerhebung

Während das vorangegangene Kapitel das Forschungsdesign und die Hypothesen definiert hat, beschreibt dieses Kapitel den konkreten Ablauf der Studie sowie die Operationalisierung der Messgrößen. Es wird detailliert dargelegt, wie die Datenerhebung organisiert wurde, um die interne Validität der Ergebnisse sicherzustellen.

4.1 Ablauf der Untersuchung

Die Studie wurde als kontrolliertes Experiment durchgeführt. Jede Sitzung folgte einem strikten Protokoll, um Störfaktoren zu minimieren. Der Ablauf gliederte sich in fünf Phasen:

1. **Onboarding (10 Min.):** Die Teilnehmenden erhielten eine Einführung in das Szenario („Sie sind neuer Entwickler im Team...“) und die Entwicklungsumgebung. Es wurde sichergestellt, dass GitHub Copilot aktiviert und funktionsfähig war.
2. **Pre-Survey (5 Min.):** Erfassung der demografischen Daten und Selbsteinschätzung zur Kompetenzeinstufung (siehe Kapitel 3).
3. **Vibe-Coding-Challenge (60 Min.):** Die Kernphase der Untersuchung. Die Teilnehmenden bearbeiteten die Aufgaben selbstständig. Eingriffe durch den Versuchsleiter erfolgten nur bei technischen Problemen (z.B. Absturz der IDE), nicht jedoch bei inhaltlichen Fragen. **Begründung des Zeitlimits:** Die Begrenzung auf 60 Minuten dient nicht nur der organisatorischen Machbarkeit, sondern fungiert als Stressor. Zeitdruck erhöht die Wahrscheinlichkeit, dass Teilnehmende auf heuristische Strategien (*System 1*) zurückgreifen, was den *Automation Bias* verstärken kann. Dies simuliert realistische Deadline-Situationen.
4. **Post-Survey (5 Min.):** Unmittelbar nach Abschluss der Bearbeitung füllten die Teilnehmenden den Fragebogen zur subjektiven Wahrnehmung aus (NASA-TLX, TAM).
5. **Debriefing (5 Min.):** Kurzes Abschlussgespräch, um offene Fragen zu klären und qualitatives Feedback einzuholen, das im Survey nicht abgebildet werden konnte.

4.2 Operationalisierung der Metriken

Um die Hypothesen zu prüfen, wurden die abstrakten Konzepte (Qualität, Effizienz, Interaktion) in messbare Indikatoren übersetzt.

4.2.1 Funktionale Korrektheit (Effectiveness)

Die funktionale Korrektheit wird binär pro Aufgabe bewertet:

- **Erfüllt (1):** Alle Akzeptanzkriterien („Done when“) sind erfüllt, Tests bestehen.
- **Nicht erfüllt (0):** Feature fehlt, ist fehlerhaft oder Tests schlagen fehl.

Die Summe der erfüllten Aufgaben ergibt den *Functional Score* (0–8).

4.2.2 Effizienz (Efficiency)

Da die Zeit auf 60 Minuten begrenzt war, wird Effizienz primär über den Fortschritt gemessen:

$$E = \frac{\text{Anzahl erfüllter Aufgaben}}{\text{Benötigte Zeit (min)}} \quad (4.1)$$

Für Teilnehmende, die alle Aufgaben vor Ablauf der Zeit lösten, steigt der Effizienzwert entsprechend.

4.2.3 Codequalität (Quality)

Die Qualität des erzeugten Codes wird anhand statischer Analyse und manuellem Review bewertet:

- **Linting-Fehler:** Anzahl der Warnungen und Fehler durch ESLint (objektiv).
- **Wartbarkeit:** Subjektive Bewertung der Lesbarkeit und Struktur (z.B. sinnvolle Variablennamen, Modularisierung) auf einer Likert-Skala (1–5). Um die Reliabilität zu sichern, orientiert sich die Bewertung an den Clean-Code-Prinzipien von Martin (2009).

4.2.4 Interaktionsqualität (Interaction Quality)

Die Interaktion mit dem Agenten wird durch quantitative Telemetrie und qualitative Beobachtung erfasst:

- **Acceptance Rate:** Das Verhältnis von akzeptierten zu angezeigten Vorschlägen (Ghost Text). Eine hohe Rate bei Low-Codern deutet auf *Automation Bias* hin.
- **Prompt Iterations:** Wie oft muss ein Prompt umformuliert werden, bis das Ergebnis passt? Dies misst die Fähigkeit zum *Prompt Engineering*.

- **Backtracking:** Wie oft wird akzeptierter Code wieder gelöscht? Ein Indikator für *Trial & Error*.
- **Prompt-Typologie:** Unterscheidung zwischen *Inline-Kommentaren* (Code-nahe Steuerung) und *Chat-Panel* (konzeptionelle Fragen).

4.3 Güte der Untersuchung (Threats to Validity)

Die Validität der Studie wird anhand der Kategorien von Cook und Campbell (1979) diskutiert.

4.3.1 Konstruktvalidität (Construct Validity)

Messen wir wirklich das, was wir messen wollen?

- **Problem:** „Kompetenz“ ist schwer zu quantifizieren.
- **Mitigation:** Triangulation der Kompetenzmessung durch Selbsteinschätzung (Pre-Survey), objektive Berufsjahre und qualitative Beobachtung während der Aufgaben.

4.3.2 Interne Validität (Internal Validity)

Können die beobachteten Effekte eindeutig der unabhängigen Variable (Kompetenz) zugeschrieben werden?

- **Selection Bias:** Die Zuteilung zu den Gruppen basierte auf Selbsteinschätzung. Um dies zu mitigieren, wurde der Algorithmus zur Klassifizierung strikt angewendet.
- **Experimenter Bias (Rosenthal-Effekt):** Die Erwartungshaltung des Versuchsleiters könnte die Teilnehmenden beeinflussen. **Gegenmaßnahme:** Standardisierte Instruktionen und minimale Interaktion während der Coding-Phase.
- **Compensatory Rivalry (John Henry Effect):** Wenn eine Gruppe sich benachteiligt fühlt, könnte sie sich mehr anstrengen. Da alle Gruppen Copilot nutzten, ist dieses Risiko gering, jedoch könnten High-Coder versuchen, „besser als die KI“ zu sein (Saretsky 1972).

4.3.3 Externe Validität (External Validity)

Sind die Ergebnisse verallgemeinerbar?

- **Hawthorne-Effekt:** Das Wissen, beobachtet zu werden, verändert das Verhalten. Dies ist in Laborstudien unvermeidbar, wurde aber durch die realistische Aufgabenstellung (Flow-Erleben) abgemildert.

- **Ecological Validity:** Die Nutzung einer Brownfield-Umgebung erhöht die Übertragbarkeit auf Unternehmenskontexte im Vergleich zu synthetischen LeetCode-Aufgaben massiv.

4.3.4 Reliabilität (Reliability)

Sind die Ergebnisse reproduzierbar?

- **Inter-Rater Reliability:** Die Bewertung der Codequalität (Wartbarkeit) unterliegt einer gewissen Subjektivität. Durch die Verwendung eines standardisierten Kriterienkatalogs (basierend auf ISO 25010) wird versucht, die Varianz zu minimieren.

4.4 Datenanalyse-Strategie

Die Auswertung erfolgt mittels eines *Mixed-Methods*-Ansatzes:

1. **Quantitative Analyse:** Vergleich der Mittelwerte von Functional Score und Effizienz zwischen den vier Gruppen. Aufgrund der kleinen Stichprobe werden deskriptive Statistiken und Visualisierungen (Boxplots) bevorzugt.
2. **Qualitative Analyse:** Inhaltsanalyse der Screen-Recordings und des Codes, um Verhaltensmuster zu identifizieren (z.B. „Trial & Error“ bei Low-Codern vs. „Review & Refine“ bei High-Codern).

Dieser methodische Aufbau stellt sicher, dass nicht nur das *Ergebnis* (Code), sondern auch der *Prozess* (Interaktion) beleuchtet wird.

5 Technische Realisierung der Testumgebung

Dieses Kapitel dokumentiert die technische Implementierung der *BuyIn Todo Sandbox*, die als experimentelle Plattform für diese Studie dient. Es wird erläutert, wie die Brownfield-Umgebung architektonisch aufgebaut ist, um realistische Unternehmensanforderungen zu simulieren, und welche technischen Maßnahmen ergriffen wurden, um eine reproduzierbare Datenerhebung zu gewährleisten.

5.1 Anforderungen an die Sandbox

Um valide Aussagen über Vibe-Coding in Unternehmenskontexten treffen zu können, musste die Testumgebung folgende Anforderungen erfüllen:

1. **Realismus (Brownfield):** Die Codebasis darf nicht leer sein („Greenfield“), sondern muss bestehende Strukturen, technische Schulden und Konventionen enthalten, die der Agent und der Mensch verstehen müssen.
2. **Komplexität:** Der Tech-Stack muss modern, aber komplex genug sein, um triviale Lösungen zu verhindern.
3. **Isolierbarkeit:** Die Umgebung muss für jeden Teilnehmenden identisch zurückgesetzt werden können (Containerisierung).
4. **Messbarkeit:** Das System muss automatisierte Tests bereitstellen, um den funktionalen Fortschritt objektiv zu messen.

5.2 Architektur und Tech-Stack: Kognitive Dimensionen

Die Sandbox ist als klassische Full-Stack-Webanwendung konzipiert. Die Architekturwahl hat direkte Auswirkungen auf die kognitive Belastung der Teilnehmenden, analysiert durch das Framework der *Cognitive Dimensions of Notation* (Green 1989).

5.2.1 Backend: Node.js & Express (Layered Architecture)

Das Backend basiert auf Node.js mit dem Express-Framework und folgt einer strikten Schichtenarchitektur:

- **Routes:** Definition der API-Endpunkte.
- **Controllers:** Verarbeitung der HTTP-Requests und Validierung.
- **Services:** Kapselung der Geschäftslogik.
- **Repositories:** Datenzugriffsschicht.

Kognitive Dimension - Viscosity (Zähigkeit): Diese Trennung erhöht die *Viscosity* für Novizen, da eine Änderung an mehreren Stellen (Datei-Hopping) durchgeführt werden muss. Für Copilot hingegen ist diese Struktur vorteilhaft, da sie klare semantische Grenzen zieht, die das Modell über den Dateinamen und Pfadkontext erkennen kann.

5.2.2 Frontend: React & Vite

Das Frontend nutzt React mit TypeScript. Die Komponenten sind funktional aufgebaut und nutzen Hooks. **Kognitive Dimension - Hidden Dependencies:** Die Verwendung von *Custom Hooks* und *Context API* erzeugt *Hidden Dependencies*. Ein Low-Coder sieht im UI-Code nicht sofort, woher die Daten kommen. Dies testet die Fähigkeit des Agenten, diese Abhängigkeiten aufzulösen, und die Fähigkeit des Nutzers, dem Agenten den richtigen Kontext zu geben.

5.3 Technische Funktionsweise von Copilot in der Sandbox

Um zu verstehen, warum die Sandbox Copilot fordert, muss die technische Funktionsweise betrachtet werden.

5.3.1 Prompt Construction und Context Retrieval

Copilot sendet nicht nur den Code vor dem Cursor an das Modell. Der *Copilot Proxy* in der IDE sammelt Kontext aus:

- **Jaccard Similarity:** Offene Tabs werden nach Ähnlichkeit zum aktuellen Code gescannt.
- **Semantic Search:** Vektorisierte Suche im Projekt (bei neueren Versionen).

In der Sandbox sind die Aufgaben so gestaltet, dass die Lösung oft in einer *anderen* Datei liegt (z.B. muss der Controller angepasst werden, um den Service zu nutzen). Wenn der Nutzer diese Datei nicht öffnet, fehlt dem Modell der Kontext. Dies misst indirekt die Kompetenz des Nutzers, dem Agenten die richtigen „Hinweise“ zu geben.

5.3.2 Token Prediction und Ranking

Das Modell generiert mehrere Vorschläge (Beams) und rankt diese nach Wahrscheinlichkeit. In einer Brownfield-Umgebung ist die Wahrscheinlichkeit für „korrekten“ Code höher, wenn er dem Stil der Umgebung entspricht. Ein High-Coder erkennt, ob der Vorschlag den Stil bricht (z.B. `var` statt `const`), ein Low-Coder nicht.

5.4 Design der Aufgaben (Workload Grading)

Die Aufgaben sind so gestaffelt, dass sie die kognitive Last sukzessive erhöhen:

Task	Beschreibung	Kognitive Anforderung
1	Persistenz (JSON)	Verstehen des Repository-Patterns (Backend).
2	User-Accounts	Einführung neuer Entitäten, Anpassung mehrerer Layer.
3	Kategorien (UI)	Frontend-Logik, State-Management.
4	Attachments	Umgang mit Binärdaten, API-Erweiterung.
5	Kalender	Komplexe Logik, Datumsberechnung (hoher Intrinsic Load).
6	Email-Verifikation	Integration externer Konzepte (Mocking).
7	Performance	Optimierung, kein funktionales Feature (Experten-Task).
8	ICS-Export	Standard-Format, reines Mapping (Erholung).

Tabelle 5.1: Aufgaben-Design und kognitive Anforderungen

5.5 Automatisierung der Evaluation

Um die Auswertung der Ergebnisse zu standardisieren, wurde ein Evaluations-Framework entwickelt.

5.5.1 Das Analyse-Skript (`run.js`)

Ein Node.js-Skript automatisiert die Prüfung der Lösungen. Es führt folgende Schritte aus:

1. **Test-Execution:** Ausführung der Unit-Tests (Vitest) und End-to-End-Tests (Playwright).
2. **Linting:** Prüfung auf Code-Style-Verletzungen mittels ESLint.
3. **Report-Generierung:** Zusammenfassung der Ergebnisse in einer JSON-Datei.

5.5.2 Grenzen der LLM-gestützten Code-Analyse

Zusätzlich zur rein funktionalen Prüfung wird ein LLM-Prompt-Template verwendet, um qualitative Aspekte des Codes zu bewerten. Hierbei muss kritisch angemerkt werden, dass LLMs zu *Halluzinationen* neigen können, also Fehler finden, die nicht existieren, oder echte Probleme übersehen. Daher dient dieser Score nur als Indikator und wird stichprobenartig manuell validiert.

5.6 Relevanz für den Unternehmenskontext (BuyIn)

Die gewählte Architektur und Methodik spiegelt reale Herausforderungen bei BuyIn wider:

- **Legacy-Integration:** Wie bei BuyIn müssen Entwickler oft in gewachsenen Strukturen arbeiten, die nicht perfekt dokumentiert sind. Vibe-Coding kann hier helfen, den „impliziten Kontext“ schneller zu erschließen.
- **Security & Compliance:** Der Einsatz von Copilot in Enterprise-Umgebungen birgt Risiken (Datenabfluss, Lizenzverletzungen). Die Sandbox simuliert dies durch Aufgaben, bei denen sensible Daten (User-Accounts) behandelt werden müssen.
- **Skalierbarkeit:** Die modulare Struktur der Sandbox zeigt, wie Copilot in größeren Teams (wo Modularisierung Pflicht ist) performt, im Gegensatz zu kleinen Skript-Projekten.

5.7 Containerisierung und Reproduzierbarkeit

Die gesamte Umgebung ist mittels Docker containerisiert. Dies stellt sicher, dass alle Teilnehmenden – unabhängig von ihrem Betriebssystem – unter identischen Bedingungen arbeiten und technische Störfaktoren (z.B. unterschiedliche Node-Versionen) eliminiert werden.

6 Ergebnisse

6.1 Datengrundlage und Toolkontext

Dieses Unterkapitel beschreibt die Datengrundlage der Untersuchung sowie den technischen Kontext der durchgeführten *VibeCoding-Challenge*. Die Analyse basiert auf einem Datensatz von $N = 18$ Probanden, die im Rahmen einer kontrollierten Coding-Challenge Aufgaben zur Entwicklung einer Todo-Applikation bearbeiteten.

6.1.1 Studiensetting und Rahmenbedingungen

Die Studie wurde unter einheitlichen Rahmenbedingungen durchgeführt, um die Vergleichbarkeit der Ergebnisse zu gewährleisten:

- **Zeitlimit:** Alle Probanden hatten ein striktes Zeitlimit von 60 Minuten für die Bearbeitung der Aufgaben.
- **Tooling:** Als Entwicklungsumgebung diente Visual Studio Code mit der GitHub Copilot Extension.
- **KI-Modell:** Als zugrundeliegendes Large Language Model (LLM) wurde **Claude 3.5 Sonnet** verwendet. Die Interaktion erfolgte über das Chat-Interface von GitHub Copilot, wobei das Modell explizit als Backend konfiguriert war.
- **Aufgabenstellung:** Die Probanden erhielten eine Liste von 8 Aufgaben (T1–T8) mit steigender Komplexität, beginnend bei Backend-Persistenz bis hin zu komplexen Frontend-Features und Export-Funktionen.

6.1.2 Datenquellen und Artefakte

Die empirische Auswertung stützt sich auf drei primäre Datenquellen, die eine Triangulation der Ergebnisse ermöglichen:

1. **Code-Artefakte (Git-Repositories):** Die finalen Implementierungen der Probanden liegen als separate Git-Branches vor. Diese Branches enthalten den vollständigen Quellcode zum Zeitpunkt des Zeitablaufs.
 - Pfad: Remote-Branches mit dem Suffix `*-vibe` (z.B. `origin/august-martin-vibe`).
2. **Chat-Logs:** Die vollständigen Interaktionsprotokolle zwischen Proband und KI-Assistent wurden exportiert. Diese Protokolle enthalten alle Prompts der Nutzer sowie die Antworten und Code-Generierungen der KI.

- Pfad: `thesis/chats/` (Format: `.docx`).
3. **Survey-Daten:** Im Anschluss an die Challenge füllten die Probanden einen Fragebogen zur Selbsteinschätzung und subjektiven Wahrnehmung der KI-Interaktion aus.
- Pfad: `thesis/surveys/Probanden-Einstufungsformular (Responses).xlsx`.

6.1.3 Dateninventar und Vollständigkeit

Tabelle 6.1 zeigt das Inventar der verfügbaren Datenartefakte. Insgesamt liegen für 18 Probanden Daten vor.

Artefakt-Typ	Anzahl	Anmerkung
Git-Banches (*-vibe)	16	2 Probanden arbeiteten lokal/ohne Push*
Chat-Logs	18	Vollständig für alle Teilnehmer
Survey-Einträge	18	Vollständig ($N = 18$)

Tabelle 6.1: Inventar der Datenartefakte. (*Für die fehlenden Remote-Banches liegen lokale Stände oder Chat-Rekonstruktionen vor, die für die Analyse genutzt wurden.)

6.1.4 Zuordnung und Zusammenführung der Daten

Die Zusammenführung der Datenquellen erfolgte über eine Mapping-Tabelle, die die Klarnamen aus dem Survey und den Chat-Dateinamen den entsprechenden Git-Banches zuordnet.

- **Survey ↔ Branch:** Das Survey-Feld "Branch-Name" sowie der Name des Probanden dienten als Schlüssel.
- **Chat ↔ Branch:** Die Dateinamen der Chat-Logs (z.B. `Max_Helling.docx`) wurden normalisiert und den entsprechenden Branches (z.B. `helling-max-vibe`) zugeordnet.

Die Survey-Daten enthalten zudem eine Einteilung in Kompetenzgruppen (No-Code, Low-Code, Mid-Code, High-Code), die in der späteren Analyse als vergleichendes Merkmal herangezogen wird.

6.1.5 Reproduzierbarkeit und Auswertungswerkzeuge

Um die Ergebnisse objektiv und reproduzierbar zu gestalten, kommen automatisierte Analysewerkzeuge zum Einsatz, deren Resultate im Ordner `evaluation/` persistiert werden:

- **Statische Code-Analyse:** Einsatz von ESLint zur Überprüfung der Code-Qualität und Einhaltung von Standards.
- **Automatisierte Tests:** Ausführung der vorhandenen Test-Suite (basierend auf Jest/Vitest), um die funktionale Korrektheit der Implementierungen (Task Completion) zu verifizieren.
- **Chat-Parsing:** Algorithmische Auswertung der Chat-Logs zur Bestimmung von AttemptedTasks und Identifikation von Prompting-Mustern.

6.1.6 Abgrenzung zu Kapitel 7

Das vorliegende Kapitel 6 beschränkt sich auf die deskriptive Darstellung der Ergebnisse und die objektive Auswertung der Daten. Eine Interpretation dieser Ergebnisse, die Diskussion von Ursache-Wirkungs-Beziehungen sowie die Beantwortung der Forschungsfragen erfolgen im anschließenden Kapitel 7 (Diskussion).

6.2 Task-Pipeline-Analyse

In diesem Abschnitt wird der Fortschritt der Probanden entlang der definierten Aufgaben-Pipeline dargestellt. Es wird zwischen versuchten, implementierten und verifizierten Aufgaben unterschieden.

6.3 Detaillierte Task-Analyse

Dieser Abschnitt bietet einen detaillierten Einblick in die Lösungsstrategien und Ergebnisse der einzelnen Aufgaben (Tasks 1 bis 5).

6.3.1 Task 1: Persistente Datenspeicherung

Hier werden die gewählten Strategien zur Datenpersistenz und deren technische Umsetzung analysiert.

6.3.2 Task 2: Benutzerauthentifizierung

Dieses Unterkapitel beleuchtet die Implementierung der Benutzerverwaltung und Authentifizierung.

6.3.3 Task 3: Kategorien und Farbcodierung

Die Ergebnisse zur Umsetzung der Kategorienverwaltung und der visuellen Farbcodierung werden hier dargestellt.

6.3.4 Task 4: Dateianhänge

In diesem Abschnitt wird die Umsetzung der Dateiupload-Funktionalität analysiert.

6.3.5 Task 5: Kalenderansicht

Hier werden die Versuche und Ergebnisse zur Implementierung der Kalenderansicht beschrieben.

6.4 Code-Qualität und Metriken

Dieses Unterkapitel präsentiert die objektiven Messwerte zur Code-Qualität, basierend auf automatisierten Analysetools.

6.5 Chat-Interaktionsmuster

Die Analyse der Interaktionen zwischen Probanden und dem KI-Modell wird in diesem Abschnitt dargestellt.

6.6 Triangulation der Datenquellen

Hier werden die Ergebnisse aus Code-Analyse, Chat-Logs und Test-Reports zusammengeführt, um Zusammenhänge sichtbar zu machen.

6.7 Zusammenfassung der Ergebnisse

Dieses Unterkapitel fasst die zentralen empirischen Befunde des Kapitels kurz zusammen.

Literatur

- Anderson, John R. (1996). *The Architecture of Cognition*. Mahwah, NJ: Psychology Press.
- Barke, Shraddha, Michael B. James und Nadia Polikarpova (2023). „Grounded Copilot: How Programmers Interact with Code-Generating Models“. In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1, S. 1–27. DOI: 10.1145/3586030.
- Bavarian, Mohammad u. a. (2022). „Efficient Training of Language Models to Fill in the Middle“. In: *arXiv preprint arXiv:2207.14255*. DOI: 10.48550/arXiv.2207.14255.
- Brandtner, Matthias, Philipp Leitner und Cor-Paul Bezemer (2024). „Challenges of Brownfield Software Modernization in Practice: A Systematic Review“. In: *Journal of Systems and Software*. DOI: 10.1016/j.jss.2024.111743.
- Collins, Allan, John Seely Brown und Susan E. Newman (1989). „Cognitive Apprenticeship: Teaching the Crafts of Reading, Writing, and Mathematics“. In: *Knowing, Learning, and Instruction: Essays in Honor of Robert Glaser*, S. 453–494.
- Cook, Thomas D. und Donald T. Campbell (1979). *Quasi-Experimentation: Design & Analysis Issues for Field Settings*. Boston: Houghton Mifflin. ISBN: 978-0395307878.
- Dreyfus, Hubert L. und Stuart E. Dreyfus (1986). *Mind over Machine: The Power of Human Intuition and Expertise in the Era of the Computer*. New York: Free Press.
- Forsgren, Nicole, Margaret-Anne Storey und Thomas Zimmermann (2021). „SPACE: A Framework for Understanding Productivity in Software Engineering“. In: *Communications of the ACM* 64.6, S. 33–36. DOI: 10.1145/3454122.
- Green, Thomas R. G. (1989). *Cognitive Dimensions of Notations*. Cambridge University Press, S. 443–460.
- Horvitz, Eric (1999). „Principles of Mixed-Initiative User Interfaces“. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, S. 159–166. DOI: 10.1145/302979.303029.
- ISO/IEC 25010:2011 *Systems and Software Engineering—System and Software Quality Models* (2011).
- Kahneman, Daniel (2011). *Thinking, Fast and Slow*. New York: Farrar, Straus und Giroux. ISBN: 978-0374275631.
- Lave, Jean und Etienne Wenger (1991). *Situated Learning: Legitimate Peripheral Participation*. Cambridge: Cambridge University Press.
- Lee, John D. und Katrina A. See (2004). „Trust in Automation: Designing for Appropriate Reliance“. In: *Human Factors* 46.1, S. 50–80. DOI: 10.1518/hfes.46.1.50.
- Martin, Robert C. (2009). *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall. ISBN: 978-0132350884.

- Mozannar, Hussein, Gagan Bansal, Adam Fourney und Eric Horvitz (2022). „Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming“. In: *arXiv preprint arXiv:2210.14306*.
- Nadi, Sarah u. a. (2022). „Human-AI Collaboration in Software Engineering: Challenges and Opportunities“. In: *arXiv preprint arXiv:2206.01081*. DOI: 10.48550/arXiv.2206.01081.
- Parasuraman, Raja und Dietrich H. Manzey (2010). „Complacency and Bias in Human Interaction with Automation: An Attentional Integration“. In: *Human Factors* 52.3, S. 381–410. DOI: 10.1177/0018720810376055.
- Pearce, Henry u. a. (2022). „Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions“. In: *IEEE Symposium on Security and Privacy*. DOI: 10.1109/SP46214.2022.9833573.
- Peng, Sida, Eirini Kalliamvakou, Peter Cihon und Mert Demirel (2023). „The impact of AI on developer productivity: Evidence from GitHub Copilot“. In: *arXiv preprint arXiv:2302.06590*. DOI: 10.48550/arXiv.2302.06590.
- Saretsky, Gary (1972). „The OEO P.C. Experiment and the John Henry Effect“. In: *Phi Delta Kappan* 53.9, S. 579–581.
- Sweller, John (1988). „Cognitive Load During Problem Solving: Effects on Learning“. In: *Cognitive Science* 12.2, S. 257–285.
- Vaithilingam, Priyan u. a. (2022). „Expectations vs. Reality: Evaluating Code Completion Tools in Realistic Settings“. In: *CHI Conference on Human Factors in Computing Systems*. DOI: 10.1145/3491102.3502031.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser und Illia Polosukhin (2017). „Attention Is All You Need“. In: *Advances in Neural Information Processing Systems* 30, S. 5998–6008.
- Witte, Thomas u. a. (2023). „Context Windows and the Limits of AI-Assisted Programming“. In: *arXiv preprint arXiv:2308.10253*. DOI: 10.48550/arXiv.2308.10253.
- Wohlin, Claes, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell und Anders Wesslén (2012). *Experimentation in Software Engineering*. Springer. ISBN: 978-3-642-29043-5. DOI: 10.1007/978-3-642-29044-2.